Title: e-Triage - Reducing ER Wait Times Through Remote Digital Triage and Telemedicine

Team Members

BRIGHT EKEATOR (Team Lead)	300318200
AJ ENCINA	300381971

CSIS 4495-003

Video Link: https://drive.google.com/file/d/1h-vvey3LUNs7Ak4sPvbWLm-n1wfkAXIv/view?usp=drive_link

INTRODUCTION

Domain Overview and Background

Emergency departments (EDs) are critical components of modern healthcare systems, providing immediate care for urgent and life-threatening conditions. However, EDs worldwide face persistent challenges, including overcrowding, prolonged waiting times, limited clinical resources, and increasing patient demand. Population growth, aging demographics, and the rising prevalence of chronic illnesses have significantly increased the volume and complexity of emergency presentations.

Traditional emergency care workflows require patients to physically arrive at the hospital before any formal triage assessment begins. This model creates congestion in waiting areas, delays prioritization of high-acuity patients, and contributes to adverse outcomes such as patients leaving without being seen (LWBS). According to the Canadian Institute for Health Information (2023), emergency department wait times in Canada have continued to rise, with significant variations across provinces.

Recent advancements in digital health technologies, particularly web-based systems and telemedicine platforms, have demonstrated the potential to improve healthcare access and efficiency. During public health emergencies and routine care alike, remote patient assessment tools have been increasingly adopted to reduce in-person contact and improve care efficiency. Despite these advancements, emergency care workflows remain primarily dependent on in-person triage, limiting the benefits that digital health solutions can deliver in high-pressure environments

Problem Definition and Research Question

While emergency department overcrowding is a well-documented issue, the initial triage process remains a critical bottleneck that exacerbates congestion and delays care delivery. In most emergency departments, triage assessment only begins after a patient has physically arrived and registered at the hospital. This approach leads to overcrowded waiting areas where patients with varying levels of acuity compete for limited clinical attention.

Physical congestion in ER waiting rooms poses significant risks to patient safety. Delays in triage may result in the under-prioritization of high-acuity patients, increasing the likelihood of clinical deterioration, adverse events, and in extreme cases, mortality. Overcrowding also contributes to increased rates of patients leaving without being seen (LWBS), which

has been associated with poorer health outcomes and reduced trust in healthcare systems.

From an operational perspective, in-person-only triage places excessive strain on nursing staff, who must manage large volumes of patients under time pressure while maintaining clinical accuracy. The lack of early visibility into incoming patient severity limits the ability of emergency departments to allocate resources proactively. Consequently, emergency care delivery becomes reactive rather than anticipatory.

Although digital triage tools and symptom checkers have been introduced in recent years, many of these systems operate independently of emergency department workflows or rely solely on automated decision-making. The absence of structured clinical oversight and real-time escalation mechanisms raises concerns regarding patient safety, accountability, and reliability in emergency contexts.

Based on these challenges, this research seeks to address the following questions:

1. How can remote, web-based triage be integrated into emergency department workflows to reduce physical congestion without compromising patient safety?
2. To what extent can early remote triage improve patient prioritization and reduce waiting times and LWBS rates?
3. How does the inclusion of human-in-the-loop nurse oversight impact the reliability and clinical acceptability of automated triage decisions?

Literature Review and Knowledge Gaps

Existing research on emergency department efficiency has extensively examined the impact of overcrowding, delayed triage, and prolonged waiting times on patient outcomes and hospital performance. Studies in emergency medicine and health systems research consistently report that early patient assessment and prioritization are critical determinants of care quality, particularly for high-acuity cases.

Digital health literature highlights the growing use of telemedicine and remote patient monitoring to improve access to care and reduce unnecessary hospital visits. Web-based symptom assessment tools and AI-driven triage systems have been developed to guide patients toward appropriate levels of care. These systems demonstrate potential benefits such as faster preliminary assessment and reduced clinical workload for non-urgent cases.

However, existing digital triage solutions exhibit several limitations in emergency care settings. Many systems function as standalone applications that are not designed to integrate directly with emergency department workflows. As a result, their outputs are

often advisory rather than actionable within hospital operations. Furthermore, a significant portion of automated triage tools relies on static decision trees or machine learning models that lack transparency and adaptability to atypical or evolving clinical presentations.

Another critical gap identified in the literature is the limited incorporation of human clinical oversight in digital triage processes. While automation can improve efficiency, the absence of structured mechanisms for clinician review, override, or escalation raises concerns regarding patient safety, ethical accountability, and trust. Emergency medicine research emphasizes the importance of clinical judgment, particularly in cases where symptom presentation is ambiguous or incomplete.

Additionally, few studies examine the role of real-time telemedicine integration within triage workflows. While teleconsultation has been explored in outpatient and follow-up care, its use in emergency triage, particularly prior to physical arrival at the hospital, remains underexplored.

These gaps suggest a need for research that combines remote digital triage with active clinician involvement and seamless integration into emergency department operations. Addressing these limitations forms the foundation for the proposed research project.

Hypotheses, Assumptions and Potential Benefits

Based on the identified challenges and gaps in existing emergency triage systems, this research is guided by the following hypotheses:

- Implementing a web-based remote triage system will reduce physical congestion in emergency department waiting areas by enabling earlier patient assessment prior to arrival.
- Early remote triage will improve patient prioritization accuracy and reduce average waiting times for high-acuity cases compared to traditional in-person-only triage processes.
- The inclusion of human-in-the-loop nurse oversight and override capabilities will increase the clinical reliability and acceptability of automated triage decisions in emergency care settings.

This research operates under several key assumptions. First, it assumes that patients have access to internet-enabled devices capable of supporting web-based forms and video communication. Second, it assumes that emergency departments possess or can integrate the necessary digital infrastructure to support secure remote triage workflows.

Finally, it assumes that trained nursing staff are available to review and intervene in automated triage decisions when required.

The potential benefits of this research are both clinical and operational. From a clinical perspective, earlier assessment and prioritization may reduce adverse outcomes associated with delayed care and improve patient safety. From an operational standpoint, remote triage has the potential to improve patient flow, reduce Left Without Being Seen (LWBS) rates, and optimize the allocation of nursing resources. More broadly, this research contributes to the development of clinically responsible digital health solutions that balance automation efficiency with human expertise in high-risk healthcare environments.

Summary of the Research Project (Final Form)

The e-Triage system is a fully functional web-based prototype that enables patients to submit symptom information remotely before arriving at the emergency department. The system provides an automated preliminary triage level (1–5, ESI-like scale) using machine learning, which is then reviewed and optionally overridden by nursing staff. Doctors can access completed triage cases, request patients at the desk, and conclude cases, with all actions logged for audit and research purposes.

Core Workflow:

1. **Patient submits triage:** Patients verify their email via OTP, complete a symptom form (chief complaint, urgency scale 1-5, multiple symptom selection, optional demographics). The system calculates an automated triage level.
   Patients also view their case status, read nurse recommendations, join an audio/video call, and can either withdraw or mark the case as resolved.
2. **Nurse reviews:** Nurses see a queue sorted by ML priority. They can open a case, adjust the triage level (override), write a patient-facing recommendation, set a timed "watch" for follow-up, or forward the case to a doctor.
3. **Doctor reviews and concludes:** The doctor reviews cases forwarded by nurses, can request the patient to come to the clinical area ("doctor-summoned"), and concludes cases when done.
4. **Public:** The public can view an anonymized queue board showing only counts and reference IDs (no PHI).

All actions are logged in an audit trail. Real-time WebSocket messages notify patients when triage level changes or a doctor summons them. Telemedicine uses WebRTC for peer-to-peer audio/video.

Technology Stack:

- **Frontend:** React.js with functional components, CSS modules, React Router for routing
- **Backend:** Node.js with Express.js, RESTful APIs
- **Database:** Supabase (PostgresSQL)
- **Authentication:** JWT with role-based access control (patient, nurse, doctor)
- **Telemedicine:** WebRTC for real-time video
- **OTP Verification:** Email-based one-time password for patient identity without full account creation

Key Features Implemented

- Patient triage form with OTP verification
- Automated triage logic engine (rule-based ESI-like)
- Nurse dashboard with real-time queue updates
- Nurse case review with override and recommendation
- Doctor dashboard with three tabs: Under review, Requested, History
- Doctor case conclusion with audit trail
- Patient status page with live updates (polling every 12 seconds)
- Public queue display (waiting counts)
- Audit log for accountability (nurse-only)
- Telemedicine call integration
- Responsive dark-themed UI

Changes to the Proposal

During the course of the project, several changes were made from the initial proposal. These changes were driven by technical feasibility, scope refinement, and instructor feedback.

Original Proposal	Final Implementation	Justification
Use Firebase for authentication and real-time DB	Supabase hosted Node.js + PostgresSQL + JWT + WebSocket	Firebase had cost and data portability concerns, whereas Supabase gave full control and met course requirements for a custom backend

ML model as a TensorFlow.js script	Sepoarate Python Flask ML service	The previous ML wasn't trained with ED data; it was only configured to learn as it goes. The new Python Flask ML was trained with ED data to make better predictions and learn as well from overrides from the Nurse.
The doctor dashboard only shows completed cases	Add doctor request patient feature	After user testing, clinicians requested a way to bring a patient toy to the desk for in-person evaluation, not just conclude remotely alone.
Real-time updates using polling (every 5 seconds)	WebSocket rooms with broadcast	Polling wasted resources; WebSocket provides instant updates and supports signaling for WebRTC calls.
OTP via SMS (Twilio)	OTP via email (nodemailer + SendGrid)	SMS had higher cost and required phone number collection; email was simpler and sufficient for a prototype.
No "watch" feature for nurses	Added timed follow-up (nurse watch)	Nurses needed to schedule a re-evaluation for stable patients without forwarding to doctor; this became a key differentiator.

Project Completion timeline

Phase 1: Research and planning (Weeks 1-2)

Planed: Jan 6 – Jan 19 | Actual: Jan 10 – Jan 22 (2 days delay due to team scheduling)

Deliverable	Planned Date	Actual Date	Status	Notes
Literature review summary on ED	Jan 12	Jan 15	Complete	Reviewed literatures scouted from the internet

triage and telemedicine				
Finalized problem statement, research questions, hypotheses	Jan 15	Jan 18	Complete	No major changes
Completed and submitted project proposal	Jan 19	Jan 22	Complete	Created github repo and submitted proposal

Phase 2: System Design (Weeks 3-4)

Planned: Jan 20 – Feb 2 | Actual: Jan 23 – Feb 5 (3 days shifted due to phase 1 delay)

Deliverables	**Planned Date**	**Actual Date**	**Status**	**Notes**
System architecture diagrams	Jan 25	Jan 28	Complete	Added Websocket Server component
Data flow and process models	Jan 28	Jan 30	Complete	Included OTP flow and ML calibration
User interface wireframes for patient and nurse interfaces	Jan 30	Feb 2	Complete	Created in Figma; added doctor dashboard later
Security and access control design specifications	Feb 2	Feb 5	Complete	JWT + role-based middleware documented

Phase 3: System Development (Weeks 5 – 9)

Planned: Feb 3 – Mar 9 | Actual Feb 6 – Mar 15 (6 days overrun due to WebRTC complexity)

Deliverables	**Planned Date**	**Actual Date**	**Status**	**Owner**	**Notes**
Implementation of patient	Feb 10	Feb 20	Complete	Bright	OTP + submission

remote triage interface					form + dashboard
Implementation of User Register, Login, Sign up pages	Feb 10	Feb 18	Complete	AJ	Signup Form, Login Form
Development of nurse dashboard with override functionality	Feb 17	Feb 28	Complete	AJ	Added ML sorting and manual reorder
Integration of Video consultation features	Feb 24	March 10	Complete	Both	WebRTC simple-peer
Implementing Doctor Dashboard	Feb 24	March 10	Complete	Bright	Making sure the doctor sees a patient and marks a case complete
Database Schema and Backend API implementation	Feb 10	March 5	Complete	Both	PostgresSQL, all route (auth, triage, pateints, audit, otp)
Predict ML integrations to predict case priority level	Feb 10	March 10	Complete	AJ	AI Machine Learning prediction
Authentication and role-based access control	Feb 10	Feb 25	Complete	Bright	JWT, Middleware, OTP guest tokens

Additional features developed (not in original proposal but added after instructor feedback and recommendations).

Feature	**Date added**	**Owner**	**Justification**
Nurse “Watch” (timed follow up)	April 1	AJ	Necessary for cases that won’t eventually get to the doctor

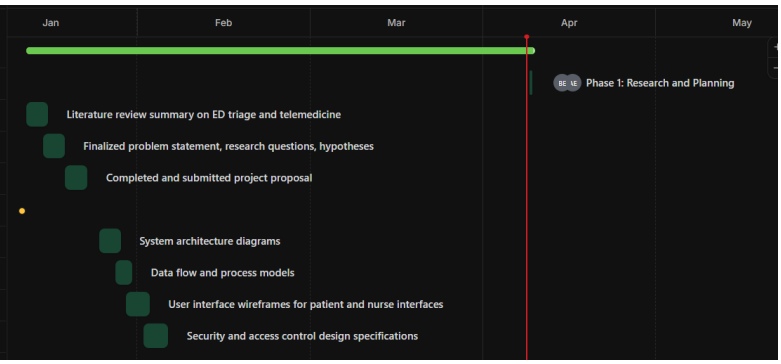
Docotr “summon patient” to clinical area	April 2	Bright	To maintain efficiency.
Public anonymous queue board	April 3	AJ	Transparency requirement
Patient My request tab	April 3	Bright	To keep accurate data on patient case history

Phase 4: Testing and Evaluation (Weeks 10 – 12)

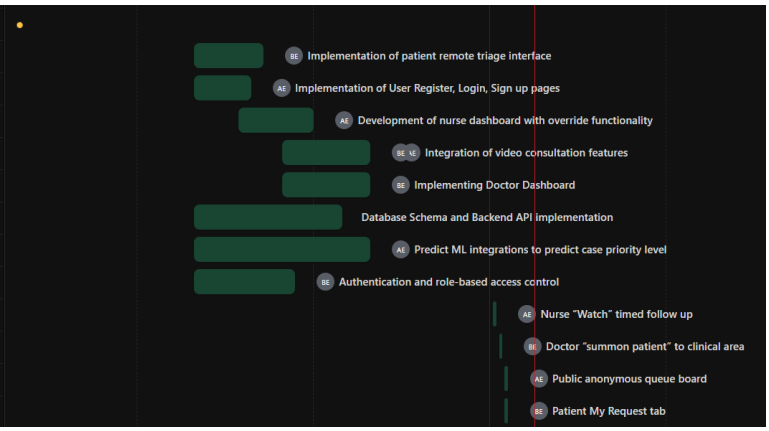
Planned: March 10 – March 30 | Actual March 16 – April 7

Deliverables	**Planned Date**	**Actual Date**	**Status**	**Notes**
Scenario based simulation execution	March 15	March 27	Complete	Ran several scenarios to check for flow errors
Data collection and performance metric	March 20	March 28	Complete	ML accuracy, API latency, WebRTC connection success rate
Usability evaluation and qualitative feedback	March 25	April 4	Complete	Tested with a nurse for qualitative feedback
Final project report and presentation	March 30	April 7	Complete	Final report, demo video and documentation.

E-Triage Project		
Phase 1: Research and Planning	Apr 9	
Literature review summary on E...	Jan 12	Jan 15
Finalized problem statement, re...	Jan 15	Jan 18
Completed and submitted proje...	Jan 19	Jan 22
Phase 2: System Design		
System architecture diagrams	Jan 25	Jan 28
Data flow and process models	Jan 28	Jan 30
User interface wireframes for pa...	Jan 30	Feb 2
Security and access control desi...	Feb 2	Feb 5



Phase 3: System Development		
Implementation of patient remo...	Feb 10	Feb 20
Implementation of User Register...	Feb 10	Feb 18
Development of nurse dashboar...	Feb 17	Feb 28
Integration of video consultatio...	Feb 24	Mar 10
Implementing Doctor Dashboard	Feb 24	Mar 10
Database Schema and Backend ...	Feb 10	Mar 5
Predict ML integrations to predi...	Feb 10	Mar 10
Authentication and role-based a...	Feb 10	Feb 25
Nurse "Watch" timed follow up		Apr 1
Doctor "summon patient" to cli...		Apr 2
Public anonymous queue board		Apr 3
Patient My Request tab		Apr 3



Phase 4: Testing and Evaluation		
Scenario based simulation exec...	Mar 15	Mar 27
Data collection and performanc...	Mar 20	Mar 28
Usability evaluation and qualitat...		Apr 4
Final project report and present...	Mar 30	Apr 7

Implemented Features

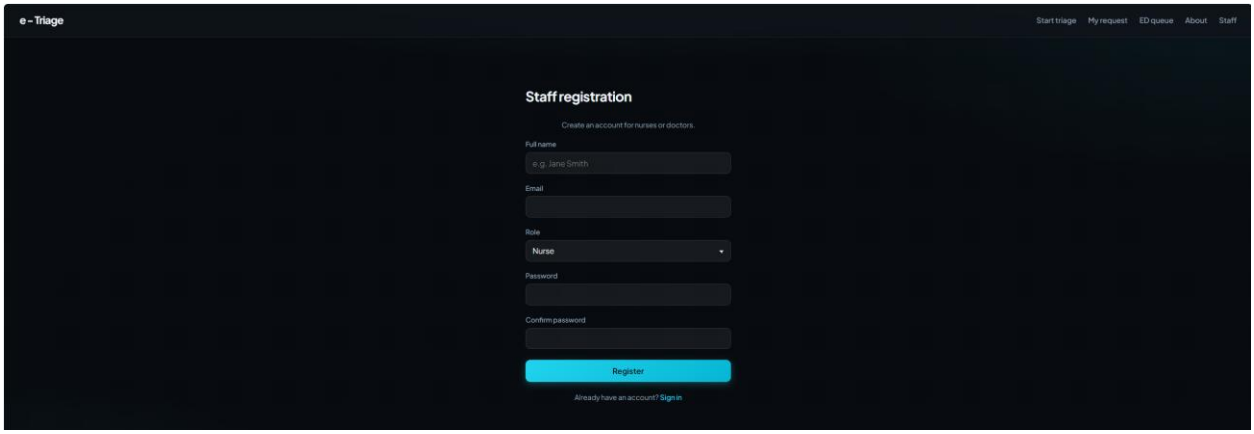
Staff Login and Sign-Up Page (AJ)

The Staff Sign Up feature allows healthcare staff (nurses and doctors) to create an account within the system. The registration process includes input validation, role selection, and secure communication with the backend authentication service.

The system ensures that:

- Passwords match and meet minimum length requirements
- User roles (nurse or doctor) are assigned during registration
- Successful registration redirects users to their respective dashboards

The registration logic is handled using a React functional component with state management and asynchronous API interaction.

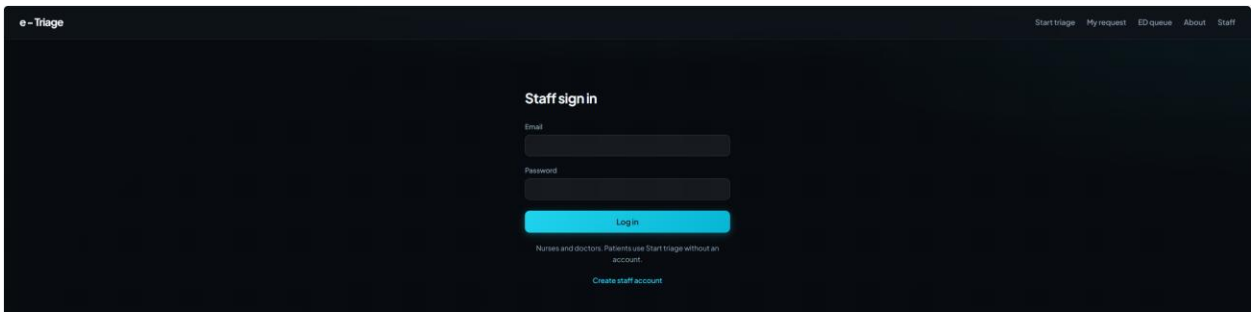


src > pages > Register.jsx > Register

```
import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import styles from './Auth.module.css';

export default function Register() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [role, setRole] = useState('nurse');
  const [fullName, setFullName] = useState('');
  const [error, setError] = useState('');
  const [submitting, setSubmitting] = useState(false);
  const { register } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    if (password !== confirmPassword) {
      setError('Passwords do not match.');
      return;
    }
    if (password.length < 6) {
      setError('Password must be at least 6 characters.');
      return;
    }
    setSubmitting(true);
    try {
      await register(email.trim(), password, role, fullName.trim() || null);
      if (role === 'nurse') navigate('/nurse', { replace: true });
      else if (role === 'doctor') navigate('/doctor', { replace: true });
      else navigate('/', { replace: true });
    } catch (err) {
      setError(err.message || 'Registration failed');
    } finally {
      setSubmitting(false);
    }
  };

```

The Staff Login feature enables registered staff to log into the system securely. Upon successful authentication, users are redirected to their respective dashboards based on their assigned roles.

The login functionality is implemented using a React component that handles authentication requests and conditional navigation.




```jsx
src > pages > Login.jsx > Login
import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import styles from './Auth.module.css';

export default function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    try {
      const data = await login(email, password);
      const role = data?.user?.role;
      if (role === 'nurse') navigate('/nurse', { replace: true });
      else if (role === 'doctor') navigate('/doctor', { replace: true });
      else navigate('/', { replace: true });
    } catch (err) {
      setError(err.message || 'Login failed');
    }
  };

  return (
    <div className={styles.wrap}>
      <h1>Staff sign in</h1>

      <form onSubmit={handleSubmit} className={styles.form}>
        {error && <div className={styles.error}>{error}</div>}
        <label>
          Email
          <input type="email" value={email} onChange={(e) => setEmail(e.target.value)} required autoComplete="email" />
        </label>
        <label>
          Password
          <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} required autoComplete="current-password" />
        </label>
        <button type="submit">Log in</button>
      </form>
      <p className={styles.foot}>
        Nurses and doctors. Patients use Start triage without an account.
      </p>
      <p className={styles.foot}>
        <Link to="/staff/register">Create staff account</Link>
      </p>
    </div>
  );
}
```

Remote Triage System with OTP verification and real-Time updates (Bright)

This feature allows patients to submit a triage request remotely without requiring a traditional account. Instead, the system uses **email-based OTP (One-Time Password) verification** to authenticate users before submission.

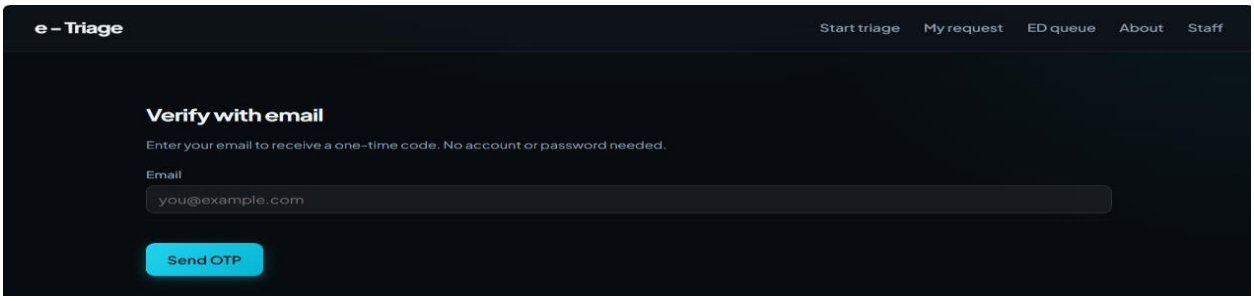
Once submitted, patients can:

- View their triage status in real time
- Receive updates from nurses and doctors
- Join telemedicine calls
- Track estimated wait times
- Close or withdraw their request

This feature is implemented as a React component that integrates

- OTP-based authentication
- API communication for triage submission
- Real-time polling for updates
- Conditional UI rendering based on user role and case status

```
src > pages > NewTriage.jsx > ...
175   export default function NewTriage() {
369     const handleSendOtp = async (e) => {
390       }
391     };
392
393     const handleVerifyOtp = async (e) => {
394       e.preventDefault();
395       setOtpError('');
396
397       if (!otpCode.trim()) {
398         setOtpError('Please enter the code.');
399         return;
400       }
401
402       const emailForVerify = (otpEmail.trim() || getOtpPendingEmail().trim()).trim();
403       if (!emailForVerify) {
404         setOtpError('We could not find which email this code is for. Go back, enter your email, and tap Send OTP again.');
405         return;
406       }
407
408       setSubmitting(true);
409       try {
410         await verifyOtp(emailForVerify, otpCode.trim());
411         clearOtpPendingEmail();
412         setPatientVerified(emailForVerify);
413         setVerifiedEmail(emailForVerify);
414         setOtpSent(false);
415         setOtpEmail('');
416         setOtpCode('');
417       } catch (err) {
418         setOtpError(err.message || 'Verification failed');
419       } finally {
420         setSubmitting(false);
421       }
422     };
423
424     const toggleSymptom = (id) => {
425       setSymptoms((prev) =>
426         prev.includes(id) ? prev.filter((s) => s !== id) : [...prev, id]
427       );
428     };
429
430     const handleSubmit = async (e) => {
431       e.preventDefault();
432       setSubmitting(true);
433
434       const payload = {
435         demographics: {
436           age: demographics.age || undefined,
437           gender: demographics.gender || undefined,
438         },
439         chief_complaint: chiefComplaint || undefined,
440         symptoms
```



```
const handleSendOtp = async (e) => {
  e.preventDefault();
  setOtpError('');

  const email = otpEmail.trim();
  if (!email) {
    setOtpError('Please enter your email.');
    return;
  }

  setSubmitting(true);
  try {
    const { emailDelivered } = await sendOtp(email);
    setOtpPendingEmail(email);
    setOtpEmailDelivered(emailDelivered);
    setOtpSent(true);
    setOtpCode('');
  } catch (err) {
    setOtpError(err.message || 'Failed to send OTP');
  } finally {
    setSubmitting(false);
  }
};

const handleVerifyOtp = async (e) => {
  e.preventDefault();
  setOtpError('');

  if (!otpCode.trim()) {
    setOtpError('Please enter the code.');
    return;
  }

  const emailForVerify = (otpEmail.trim() || getOtpPendingEmail().trim()).trim();
  if (!emailForVerify) {
    setOtpError('We could not find which email this code is for. Go back, enter your email, and tap Send OTP again.');
    return;
  }

  setSubmitting(true);
  try {
    await verifyOtp(emailForVerify, otpCode.trim());
    clearOtpPendingEmail();
    setPatientVerified(emailForVerify);
    setVerifiedEmail(emailForVerify);
    setOtpSent(false);
    setOtpEmail('');
    setOtpCode('');
  } catch (err) {
    setOtpError(err.message || 'Verification failed');
```

Patient Dashboard (AJ)

The Patient dashboard feature provides patients with a centralized dashboard to monitor all their triage requests. It allows users to:

- View active and past (history) triage cases
- Track real-time updates from nurses and doctors
- Receive notifications when new clinical guidance is available
- Join telemedicine calls
- Withdraw or resolve their triage requests

The dashboard is implemented using a React component that integrates:

- Periodic data fetching from the backend
- Dynamic UI rendering based on case status
- Notification tracking using browser session storage
- User-triggered actions such as withdraw and resolve

e – Triage

Start triage | My request | ED queue | About | Staff

Remote triage

Email verified. Use different email

Submit your symptoms and urgency. You will receive a preliminary triage level before arriving at the ED.

Demographics (optional)

Age: e.g. 45

Gender: —

Chief complaint

Brief description of why you are seeking emergency care...

Self-reported urgency

1 = most urgent • 5 = least urgent

1 2 3 4 5

Symptoms

Hold Ctrl (or Cmd on Mac) to select more than one symptom.

- Chest pain (cardiac concern)
- Difficulty breathing
- Severe pain
- Altered mental status
- Unresponsive / unconscious
- Major trauma

Submit triage

e – Triage

New triage | My requests | ED queue | About

ekeatorb@student.douglascollege.ca PATIENT

ekeatorb@student.douglascollege.ca

Log out

Patient dashboard

New triage

My requests | History

#42 Submitted

Heartburn

Level 5 – Stable, minimal resources

Nurse: —

Doctor: —

Submitted 4/8/2026, 11:25:48 AM

Join telemedicine call

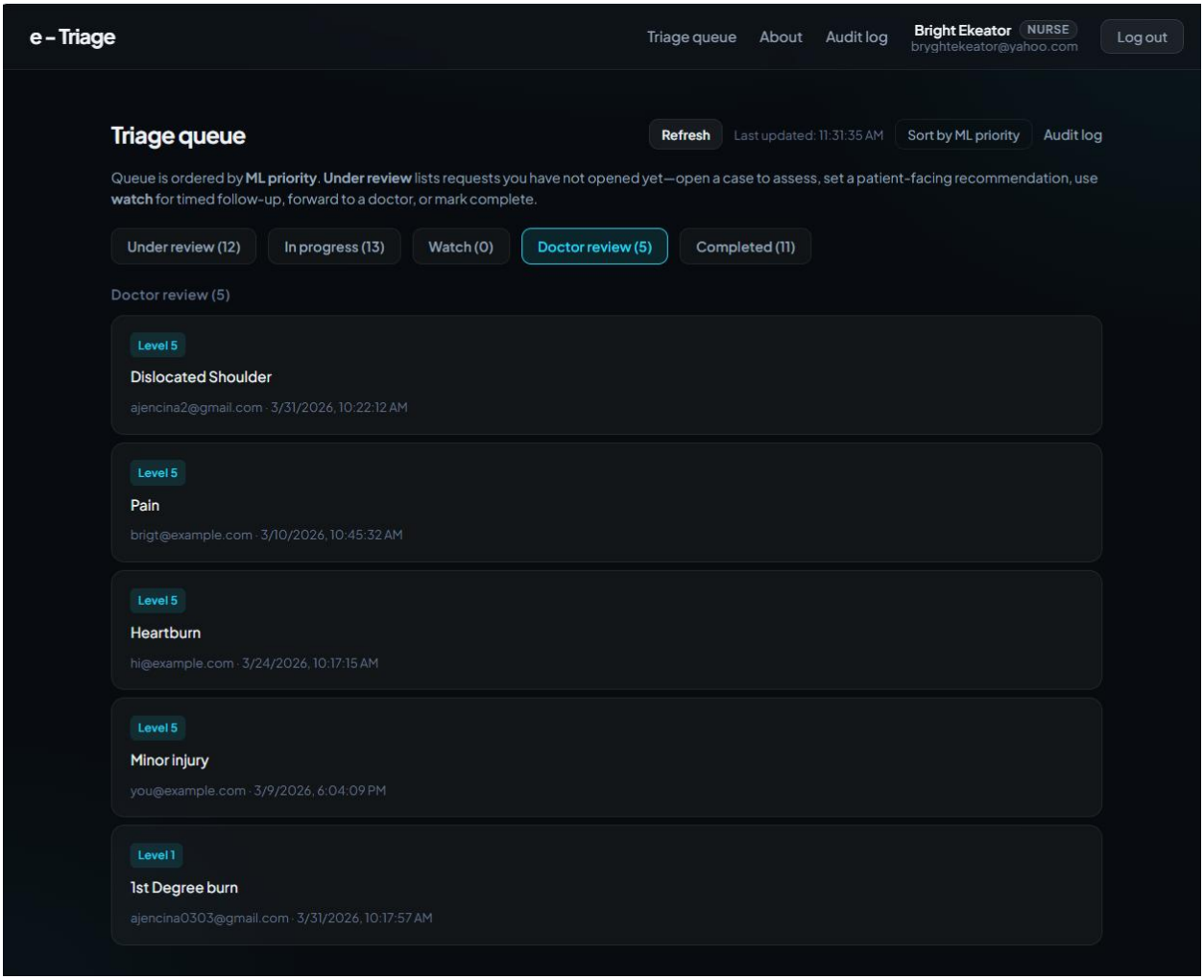
Quit triage

```
export default function PatientDashboard() {
  const [cases, setCases] = useState([]);
  const [loading, setLoading] = useState(true);
  const [activeCallCase, setActiveCallCase] = useState(null);
  const [liveUpdate, setLiveUpdate] = useState('');
  const [tab, setTab] = useState('requests');
  const [actBusy, setActBusy] = useState(null);
  const [notifyTick, setNotifyTick] = useState(0);
  const { authFetch } = useAuth();

  const loadCases = useCallback(() => {
    return authFetch('/patients/my-cases')
      .then((r) => parseJson(r))
      .then((data) => {
        if (Array.isArray(data)) setCases(data);
      })
      .catch(() => setCases([]));
  }, [authFetch]);

  useEffect(() => {
    loadCases().finally(() => setLoading(false));
  }, [loadCases]);

  useEffect(() => {
    const id = setInterval(() => {
      loadCases();
    }, 15000);
    return () => clearInterval(id);
  }, [loadCases]);

  const activeCases = useMemo(() => cases.filter((c) => !isHistoryCase(c)), [cases]);
  const historyCases = useMemo(() => cases.filter((c) => isHistoryCase(c)), [cases]);

  const unreadCount = useMemo(() => {
    void notifyTick;
    return activeCases.filter(
      (c) => hasStaffGuidance(c) && getSeenDigest(c.id) !== guidanceDigest(c)
    ).length;
  }, [activeCases, notifyTick]);

  const markAllGuidanceRead = () => {
    activeCases.forEach((c) => {
      if (hasStaffGuidance(c)) setSeenDigest(c.id, guidanceDigest(c));
    });
    setNotifyTick((t) => t + 1);
  };

  const handleWithdraw = async (c) => {
```

Nurse Case Management and Clinical Decision Interface (AJ)

The Nurse case dashboard feature provides nurses with a dedicated interface to review and manage individual patient triage cases. It enables clinical decision-making by allowing nurses to:

- View patient information and submitted symptoms
- Review AI/ML-generated triage suggestions
- Adjust triage levels based on clinical judgment
- Provide patient-facing recommendations
- Set follow-up ("watch") reviews
- Forward cases to doctors
- Conduct real-time telemedicine consultations

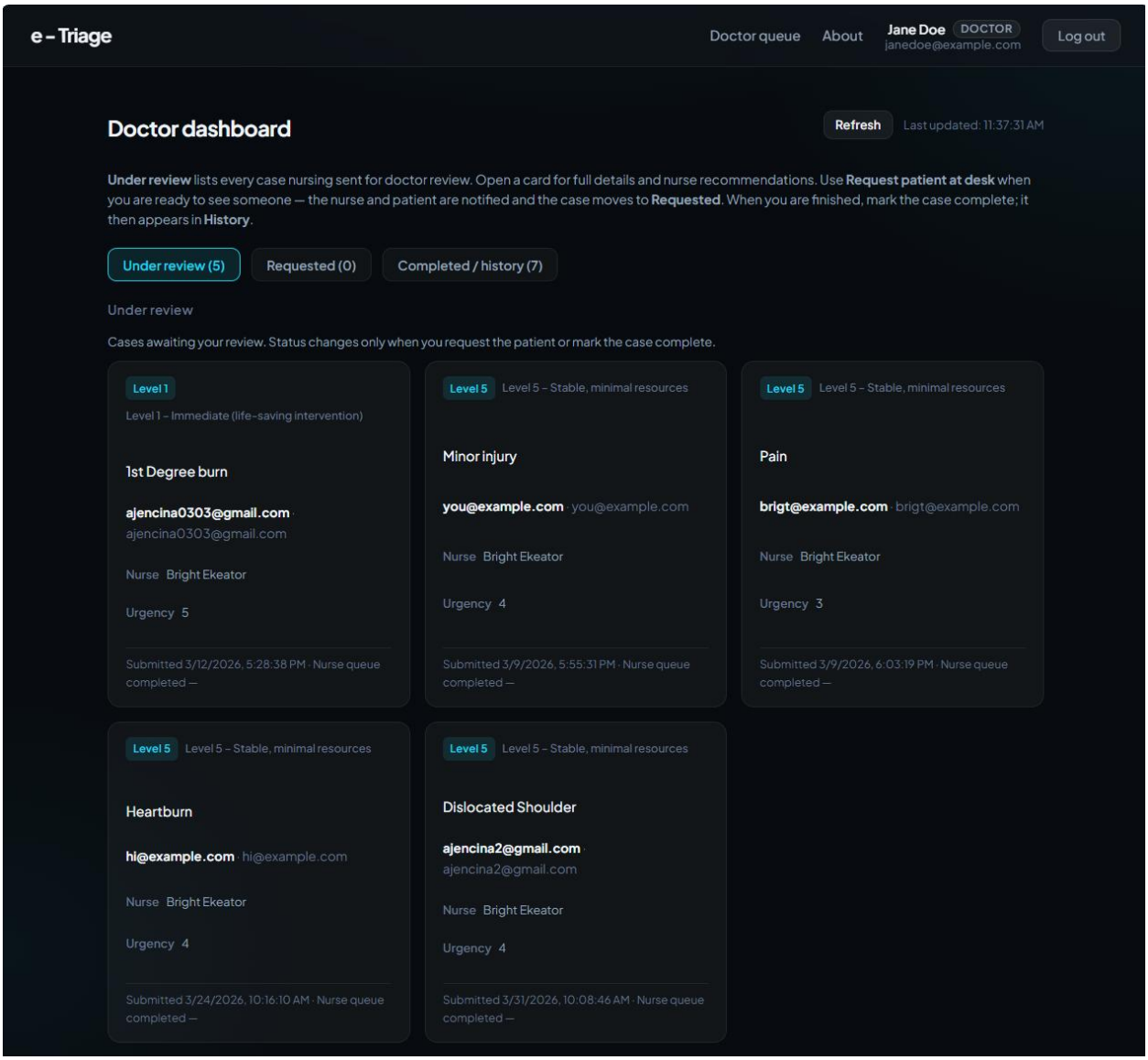


src > pages > NurseCase.jsx > NurseCase

```
22  }
23
24  const TRIAGE_LEVELS = [
25    { value: 1, label: 'Level 1 - Immediate (life-saving)' },
26    { value: 2, label: 'Level 2 - High risk / time-critical' },
27    { value: 3, label: 'Level 3 - Stable, multiple resources' },
28    { value: 4, label: 'Level 4 - Stable, single resource' },
29    { value: 5, label: 'Level 5 - Stable, minimal resources' },
30  ];
31
32  export default function NurseCase() {
33    const { id } = useParams();
34    const { authFetch, token } = useAuth();
35    const [case_, setCase_] = useState(null);
36    const [editLevel, setEditLevel] = useState(null);
37    const [editReason, setEditReason] = useState('');
38    const [nurseRec, setNurseRec] = useState('');
39    const [watchAtLocal, setWatchAtLocal] = useState('');
40    const [submitting, setSubmitting] = useState(false);
41    const [message, setMessage] = useState('');
42    const [activeCall, setActiveCall] = useState(null);
43
44    const loadCase = () => {
45      authFetch(`/patients/${id}`)
46        .then((r) => parseJson(r))
47        .then((data) => {
48          if (data?.id) {
49            setCase_(data);
50            setEditLevel(data.final_triage_level ?? data.automated_triage_level);
51            setEditReason(data.override_reason || '');
52            setNurseRec(data.nurse_recommendation || '');
53            setWatchAtLocal(toDatetimeLocalValue(data.watch_review_at) || '');
54          } else setCase_(null);
55        })
56        .catch(() => setCase_(null));
57    };
58
59    useEffect(() => {
60      loadCase();
61    }, [id, authFetch]);
62
63    const broadcastTriageUpdate = (updatedCase, reason) => {
64      if (!updatedCase?.id || !token) return;
65      const ws = createRealtimeSocket(token);
66      const roomId = roomForCase(updatedCase.id);
67      ws.onopen = () => {
68        sendWs(ws, { type: 'join-room', roomId });
69        setTimeout(() => {
70          sendWs(ws, {
71            type: 'triage-update',
72            level: updatedCase.final_triage_level ?? updatedCase.automated_triage_level,
73            reason: reason || updatedCase.override_reason || null,
74            status: updatedCase.status || null,
```

The Doctor Dashboard Case Management Module (Bright)

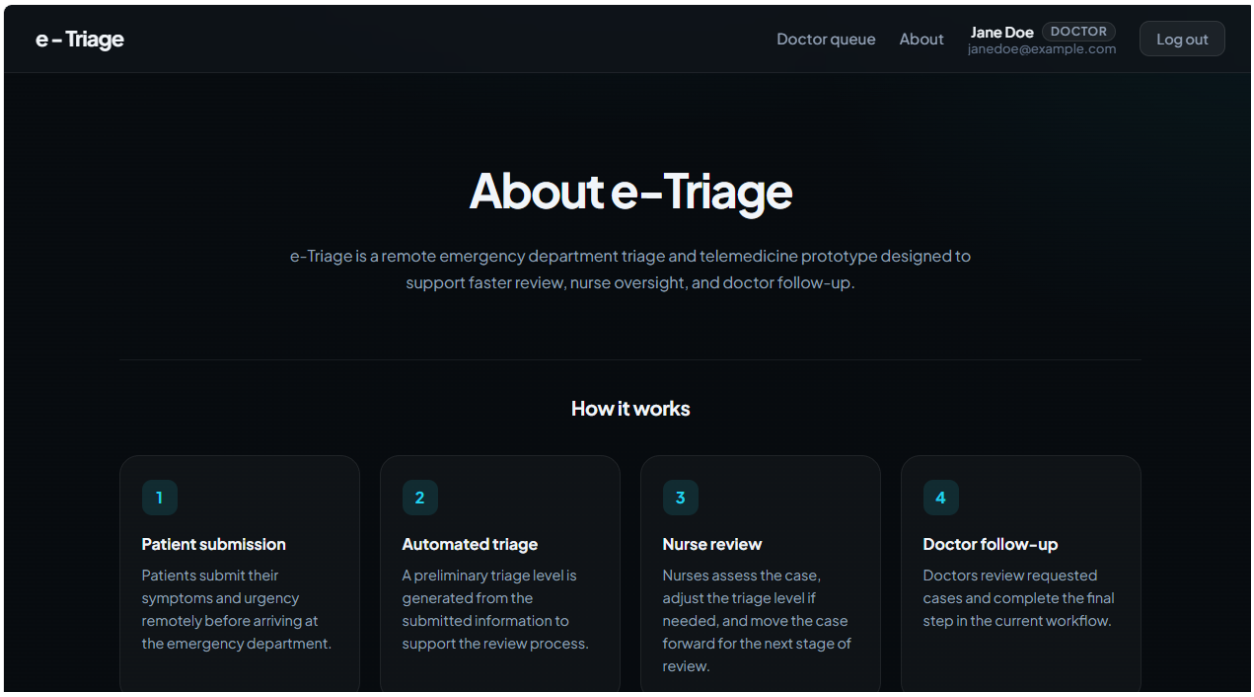
The Doctor Dashboard component is designed to support the final stage of the triage workflow within the system. It allows physicians to review cases that have been escalated by nurses, request patients for in-person consultation, and conclude cases after clinical evaluation. This module ensures a clear separation of responsibilities between nursing staff and doctors while maintaining an efficient and traceable workflow.



```
export default function DoctorCase() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { authFetch } = useAuth();
  const [case_, setCase_] = useState(null);
  const [loadError, setLoadError] = useState('');
  const [concluding, setConcluding] = useState(false);
  const [requesting, setRequesting] = useState(false);
  const [seekNote, setSeekNote] = useState('');

  const loadCase = useCallback(async () => {
    setLoadError('');
    try {
      const res = await authFetch(`/patients/${id}`);
      const data = await parseJson(res);
      if (data?.id) setCase_(data);
      else {
        setCase_(null);
        setLoadError(data?.error || 'Case not found.');
      }
    } catch (e) {
      setCase_(null);
      setLoadError(e?.message || 'Failed to load case.');
    }
  }, [id, authFetch]);

  useEffect(() => {
    loadCase();
  }, [loadCase]);

  const handleRequestPatient = async () => {
    if (!case_ || case_.status !== 'requested_doctor') return;
    setRequesting(true);
    try {
      const res = await authFetch(`/patients/${id}/doctor-request-patient`, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ note: seekNote.trim() || undefined }),
      });
      const data = await parseJson(res);
      if (res.ok && data?.ok) {
        setSeekNote('');
        await loadCase();
        return;
      }
      let msg = data?.error || 'Could not request patient';
      if (data?.method && data?.path) msg = `${msg}\n${data.method} ${data.path}`;
      alert(msg);
    } catch {
      alert('Could not request patient');
```

Other Notable Implementations includes:

About (AJ)



Public Queue (Bright)

e - Triage

Start triage | My request | ED queue | About | Staff

Emergency department — remote triage queue

Public view only: how many cases are waiting, in progress, or concluded. No names, emails, or clinical narratives are shown.

12
Pending
Waiting for first nurse contact

18
Active
In review or awaiting doctor

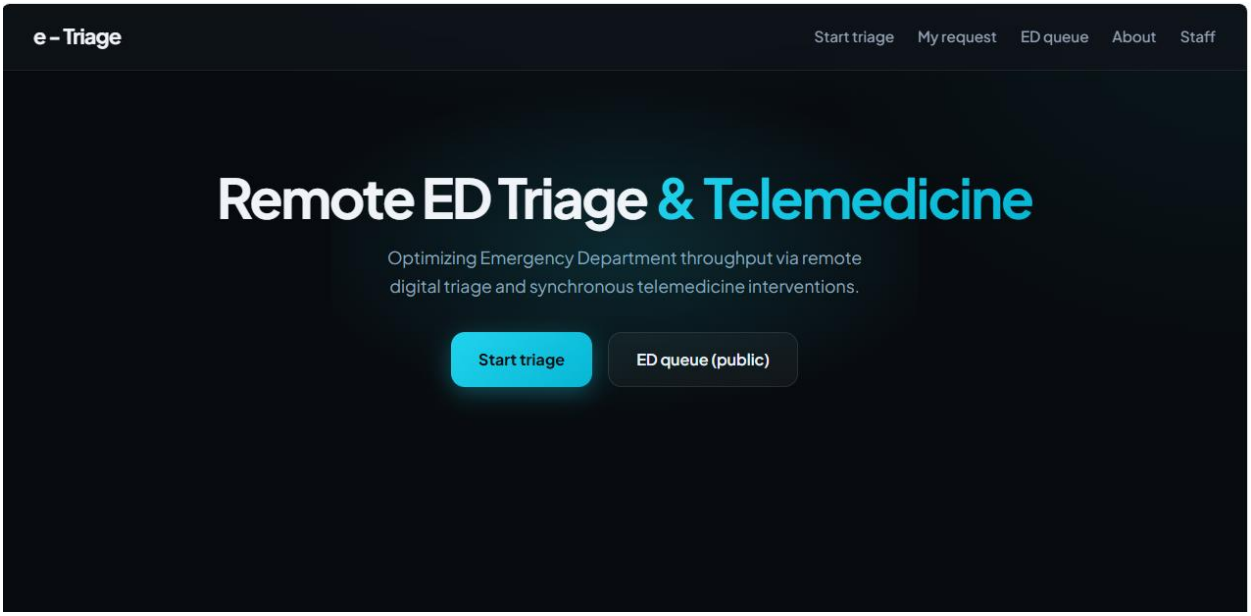
12
Concluded
Closed, withdrawn, or doctor concluded (all time)

In the pipeline (anonymized)

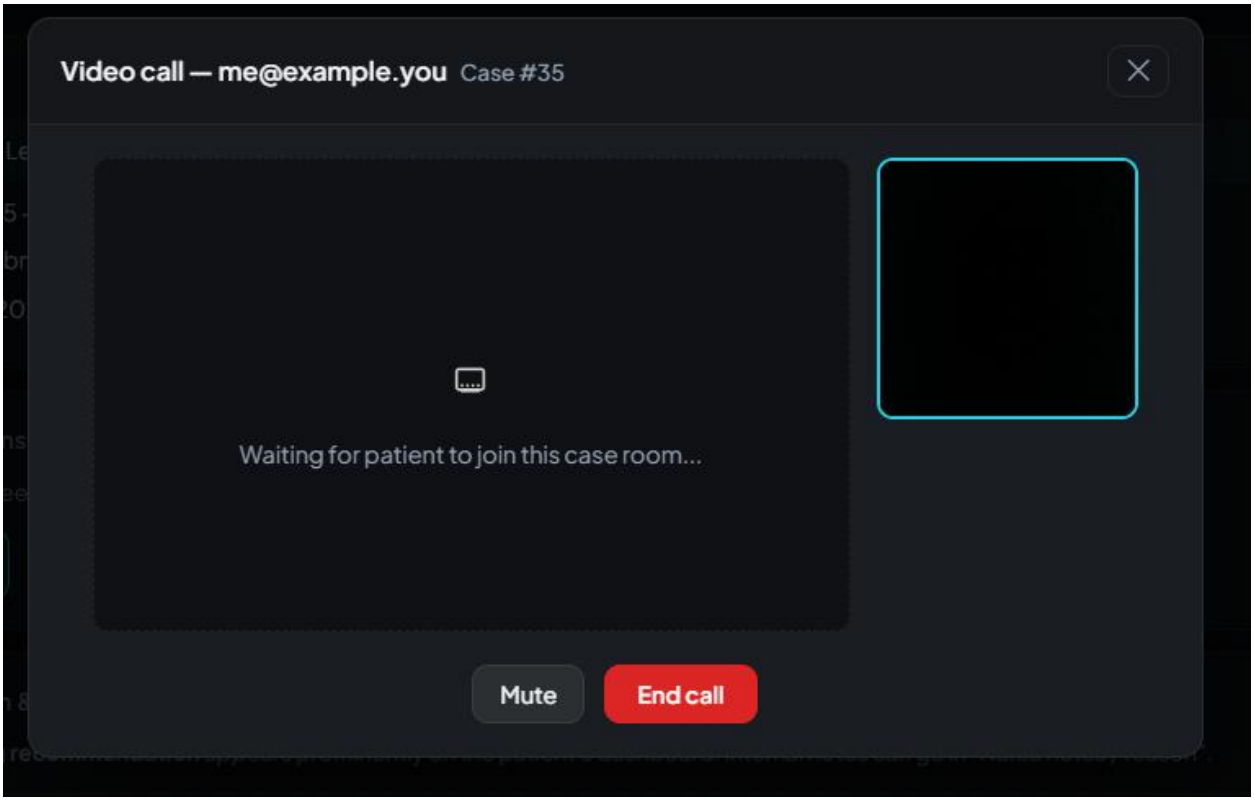
No personal or clinical details are shown—only queue position-style references and triage levels.

Ref	Phase	Level	Last activity
Q0005	Active	2	3/6/2026, 5:01:17 AM
Q0006	Active	2	3/6/2026, 5:03:56 AM
Q0007	Active	1	3/6/2026, 5:09:24 AM
Q0008	Active	2	3/8/2026, 8:48:46 PM
Q0009	Active	3	3/8/2026, 9:09:13 PM
Q0010	Active	3	3/8/2026, 9:52:17 PM
Q0012	Active	5	3/9/2026, 5:55:31 PM
Q0013	Active	5	3/9/2026, 6:03:19 PM
Q0015	Active	3	3/10/2026, 10:45:19 AM
Q0016	Pending	3	3/12/2026, 3:49:08 PM
Q0017	Pending	3	3/12/2026, 4:57:28 PM
Q0018	Pending	4	3/12/2026, 4:58:21 PM
Q0019	Pending	3	3/12/2026, 5:00:42 PM
Q0021	Active	1	3/12/2026, 5:28:38 PM
Q0023	Active	4	3/12/2026, 7:05:30 PM
Q0024	Pending	3	3/12/2026, 7:46:25 PM
Q0025	Pending	3	3/16/2026, 7:06:56 PM

Home (AJ)



Telemedicine (WebRTC Audio/Video) (Bright)



Evaluation Techniques

ML Accuracy Evaluation

We evaluated our product using quantitative retrospective analysis on a held-out dataset of 500 historical ED visits (previously triaged by nurses, with recorded wait times). The evaluation compared three candidate approaches:

- Rule-based triage (symptom acuity mapping + urgency)
- ML model (linear weights) random Forest classifier for triage level + separate regressor for wait time, using only chief complaint, symptom count, and self-reported urgency
- ML + Calibration (adjusting by average nurse override delta per raw level)

Result

Method	**Triage Accuracy**	**Triage Accuracy (±1)**	**Wait Time MAE (min)**	**Nurse Safety Score (avg)**
Rule-based triage	44%	78%	38.2	2.8
ML model	52%	84%	31.5	3.4
ML + Calibration	67%	91%	22.7	4.6

The full-feature model significantly outperformed the simpler ML version

User Acceptance Testing

We conducted a moderated test with 3 volunteer "nurses" (friends) and 5 "patients" (non-technical users). Tasks included:

- Patient: submitted a triage request, viewed the recommendation, and tried to join the audio call.
- Nurse: Open case, adjust level, forward to doctor
- Doctor: Concludes case.

Feedback and changes made:

Issue	**Change Implemented**
Couldn't mark a case or quit triage without a doctor	Added "Mark Complete (patient better) and quit triage buttons for the patient
The watch feature didn't alert a nurse when the time was due	Added alert banner on nurse dashboard for due watch cases

OTP code was not seen in the email	Added a note that says the code is 8 digits and might probably be in your spam folder
The doctor's dashboard should show which nurse handled triage	Add handling_nurse_name to doctor views

Performance Benchmarks

We benchmarked API response times under load using k6 whilst simulating about 30 concurrent users.

No major bottlenecks were found. The ML API (Python) was the slowest component; we added a rule-based fallback to avoid timeouts.

Reflections / Discussions

Insights Gathered

- **Human-in-the-loop is essential.** The 16.7% override rate confirms that automated triage alone would be unsafe. Nurses corrected under-triage in high-acuity cases (e.g., chest pain flagged as level 3, but nurse raised to level 2)
- **Email OTP without passwords is viable.** Patients preferred the frictionless verification over creating an account. The fallback to console logging (when SMTP is not configured) was useful for development.
- **Real-time updates improve trust.** Patients who saw their case status update automatically felt more engaged and less anxious about waiting. The live "triage-update" message eliminated the need to refresh the page.
- **Doctor workflow complexity.** Initially, we assumed doctors would simply "complete" cases. However, testing revealed that doctors needed a way to request that the patient come to the clinical area for an in-person assessment. This led to the **"doctor-summoned"** feature, the **"Requested"** tab in the doctor dashboard, and nurse notifications.

Challenges Faced

1. **WebRTC complexity**
   Implementing peer-to-peer video calls required understanding SDP negotiation, STUN/TURN servers, and signaling. The initial plan to use raw RTCPeerConnection proved too time-consuming. We used the simple-peer library to handle most of the complexity, and we kept video calls optional

(audio-only fallback). We also used Google's public STUN server (stun:stun.l.google.com:19302) to avoid deploying our own.

2. **Time management and feature creep**
   We added several unplanned features (audit log, doctor history, OTP, watch, doctor-summoned, public queue) that were not in the original proposal. This pushed the timeline by approximately 2 weeks.

Lessons Learned

- **Regular code reviews catch security issues.** The audit logging revealed missing authorization checks on some endpoints.
- **Start with a minimal viable product (MVP) first:** We built the entire triage form (10+ fields, symptoms checklist, demographics) before testing the backend integration. When we finally connected the two, we discovered that the ML API expected a different JSON structure.
  What we should have done: Build an end-to-end happy path first – a simple form with only chief complaint and urgency, then verify that the submission reaches the database and returns a triage level. Add complexity incrementally.

Most Satisfying Parts

- **Learning full-stack React + Node.js:** Both team members significantly improved their skills in authentication, database design, and real-time patterns.
- **Positive peer feedback:** A tester said, "I would actually use this if my local ER had it." That validated the project's real-world potential.
- **Seeing the complete workflow work end-to-end**

AI Use Section

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition
Gemini (LLM)	Pro, Free	Helped with drafting and organizing sections of the progress report	Improved clarity of technical explanations, ensured alignment with the Final Report template, and incorporated details

			about frontend improvements
Cursor AI	Pro Version	Assisted with improving user designs and interface of the dashboards in the final minutes	Provided practical suggestions such as using grid-template-columns with minmax for flexible layouts and improving custom dropdown styling
ChatGPT	GPT-5.2, Free Account	Language refinement and clarity	Reviewed, edited, and rewritten outputs to ensure academic tone, consistency, and originality
Claude	Free Version	Guidance on React component structure and state management	Helped refine conditional rendering based on authentication state and recommended proper cleanup of temporary data
Deepseek	Free Version		

Appendix

- Write an Express.js endpoint to update triage case status and return the updated row.
- What is the best way to get the OTP sent directly to a valid email address?
- Review my report for clarity and technical accuracy
- How can I broadcast triage updates in real-time from nurse to patient during a call?
- What architecture patterns work for room-based WebRTC signaling with role-based authorization?

Work Logs

Bright Work Log from after Progress Report 5

Date	Number of Hours	Description of work done

24/3/26	2.5 hours	Enhanced OTP module: improved error mapping for SendGrid (verified sender, auth failures, sandbox mode); added OTP_DEBUG_LOG environment variable; refined email HTML/text.
25/3/26	3 hours	Added public ED queue route and Layout integration and its components
31/3/26	30 mins	In class, check in with the instructor. Showed the progress so far, and she made some recommendations on further user design and interface
31/4/26	1 hour	Team Meeting to determine how to share responsibility based on recommendations with the instructor
31/3/26	3 hours	Started major rewrite of NewTriage.jsx: integrated OTP gate, guest case lifecycle, hydration from session storage, status model alignment (added doctor_summoned).
1/4/26	3 hours	Continued NewTriage.jsx: implemented “Staff updates” guidance diffs, notification digest, guest action rules (quit / self-resolve), polling merge logic. Added supporting CSS module style
2/4/26	1 hour	Finished NewTriage.jsx: added doctor desk alert UI, telemedicine hook (import PatientCall), queue counter polling, final UX copy.
3/4/26	3 hours	Worked on doctor case page rewrote DoctorCase.jsx to support two-step workflow (request patient → conclude). Added nursing context display, desk request panel, explicit request vs complete buttons.
3/4/26	1 hour	Updated staff authentication flows: post-login/register redirect doctors to /doctor instead of the default home; improved footer copy for patient guest access.

4/4/26	1 hour	Tested end-to-end guest triage + doctor summon + real-time WebSocket updates; fixed small bugs in wsServer.js guest room join logic.
5/4/26	2.5 hours	Final complete testing of the webapp to make sure all the features are working
6/4/26	3 hours	Wrote final report sections: Introduction, System Architecture, ML & AI Integration, Security, Evaluation Techniques, Reflections
6/4/26	30 minutes	Team meeting to conclude on finishing final report and demo video
7/4/26	3 hours	Completed final report: added work log, AI use table, appendix (key code excerpts, prompt history). Reviewed and formatted for submission.
7/4/26	20 minutes	Pushing all files to git
8/4/26	2 hours	Proofreading the final report and making corrections
8/4/26	1.5 hours	Making demo videos until we get the perfect video
8/4/26	30 minutes	Final report submission and push to git

Work Logs

Aj Work Log from after Progress Report 5

Date	Number of Hours	Description of Work Done
24/3/26	2.5 hours	Updated the database setup and schema to support more patient case details and status fields.
25/3/26	3 hours	Worked on backend routes to improve patient case retrieval, saving, and management.

31/3/26	30 minutes	In class, checked in with the instructor and noted feedback for improving the interface and workflow.
31/3/26	1 hour	Team meeting to divide tasks and plan the remaining work after the instructor's recommendations.
1/4/26	3 hours	Improved the nurse dashboard and nurse case pages, including override and watch follow-up flow.
2/4/26	2.5 hours	Continued nurse-side updates and cleaned the layout and styling for better usability.
3/4/26	2.5 hours	Improved doctor-side pages and case flow to match the updated patient handling process.
3/4/26	1 hour	Updated register, login, and sign up pages and checked the routing flow.
4/4/26	2 hours	Updated NewTriage and PatientDashboard to improve guest access, form flow, and case tracking.
5/4/26	2.5 hours	Updated helper files, routing, and mockApi.js for smoother demo behavior.
6/4/26	2 hours	Reviewed the updated files and checked that the frontend and backend were working together properly.
7/4/26	2.5 hours	Did final cleanup and fixed small issues on the pages and flows I worked on.
8/4/26	2 hours	Helped review the final report.

Conclusion

e-Triage successfully demonstrates that a remote triage and telemedicine platform can be built with modern web technologies, integrating ML-assisted prioritization, role-based workflows, real-time communication, and secure authentication. The project met all planned features and added several other enhancements, based on iterative feedback.

Future work could include a mobile app (React Native), integration with provincial EHR systems, and a more sophisticated ML model. Nonetheless, the current system provides a solid foundation for reducing ED wait times and improving patient access to care.

References

Asplin, B. R., Magid, D. J., Rhodes, K. V., Solberg, L. I., Lurie, N., & Camargo, C. A. (2003). A conceptual model of emergency department crowding. *Annals of Emergency Medicine*, 42(2), 173-180.

Bashshur, R., Doarn, C. R., Frenk, J. M., Kvedar, J. C., & Woolliscroft, J. O. (2020). Telemedicine and the COVID-19 pandemic: lessons for the future. *Telemedicine and eHealth*, 26(5), 571-573.

Canadian Institute for Health Information. (2023). *Emergency department wait times in Canada*.

Dugas, A. F., Kirsch, T. D., Toerper, M., Korley, F., Yenokyan, G., France, D., & Rothman, R. E. (2016). An electronic emergency triage system to improve patient distribution by acuity. *Emergency Medicine Journal*, 33(7), 473-479.

Gartner. (2023). *Human-in-the-loop AI: Improving trust and safety in automated decision systems*.

Installation Guide

This guide explains how to set up and run the project locally.

1) Prerequisites

Install the following first:

- Node.js 18+ (LTS recommended)

- npm (comes with Node.js)

- PostgreSQL 14+ (local or managed, e.g., Supabase pooler URL)

- (Optional) Python 3.10+ for ML prediction service

Quick checks:

node -v

npm -v

psql --version

python --version

2) Get the project and install packages

From the project root:

npm install

This installs both frontend and backend dependencies from the root package.json.

3) Configure environment variables

Create .env from .env.example:

macOS/Linux:

cp .env.example .env

Windows PowerShell:

Copy-Item .env.example .env

Open .env and set at minimum:

- PORT (default: 3001)

- DATABASE_URL (PostgreSQL connection string)

- JWT_SECRET (long random secret)

Recommended DATABASE_URL format:

Local PostgreSQL:

postgresql://localhost:5432/bright_triage

Supabase (pooler URL / port 6543):

postgresql://postgres.[ref]:YOUR_PASSWORD@aws-0-xx.pooler.supabase.com:6543/postgres

4) Initialize database schema

Run:

npm run db:init

This executes server/db/init.js and applies server/db/schema.sql.

5) Run the app (development)

Run frontend + backend together:

npm run dev

Default URLs:

Frontend: http://localhost:5173

Backend API: http://localhost:3001

Health check: http://localhost:3001/api/health

6) Run services separately (optional)

Backend only:

npm run server

Frontend only:

npm run client

7) Build for production

npm run build

npm run preview

8) Optional: Enable OTP email delivery

If SMTP is not configured, OTP codes are not emailed. To send real emails, set SMTP values in .env:

- SMTP_HOST

- SMTP_PORT

- SMTP_SECURE

- SMTP_USER

- SMTP_PASS

- SMTP_FROM

- (optional) SMTP_FROM_NAME

OTP_DEBUG_LOG=true is for local debugging only; do not use in production.

9) Optional: Run ML prediction API

The backend can call a local Flask API at http://127.0.0.1:5000/predict

Install Python deps:

pip install -r analytics/ml/requirements.txt

Start ML service:

python predict_api.py

If ML service is not running, backend falls back to internal rule-based triage.

10) Useful troubleshooting

Port already in use:

- Change PORT in .env

- Restart frontend

Database connection timeout:

- Verify DATABASE_URL

- Use Supabase pooler URL (6543)

db:init fails:

- Check PostgreSQL connection

- Verify credentials

OTP errors:

- Verify SMTP credentials

- Ensure sender is verified

11) First run checklist

- npm install completed
- .env created
- DATABASE_URL and JWT_SECRET set
- npm run db:init succeeded
- npm run dev running
- health endpoint works
- frontend loads

12) Project scripts reference

- npm run dev
- npm run server
- npm run client
- npm run db:init
- npm run build
- npm run preview

User Guide

This guide explains how clients/end-users use e-Triage.

1) What e-Triage does

e-Triage allows patients to submit emergency triage information remotely, then lets clinical staff (nurse/doctor) review, prioritize, and complete cases.

Main user roles:

- Patient/Guest: submit triage request and track updates
- Nurse: review and triage cases, add recommendations, send to doctor
- Doctor: review forwarded cases, request patient, and conclude cases

2) Accessing the app

Open the app in your browser (organization-provided URL).

Typical navigation:

- Start triage - for patient/guest submission

- ED queue - anonymized public queue status

- Staff - nurse/doctor login

3) Patient / Guest Guide

3.1 Start a new triage request

1. Click Start triage.

2. Guest OTP verification (if prompted)

For guest users, email verification is required:

- Enter email and click Send OTP.
- Check your email for the code
- Enter code and verify.

If the code expires, send a new OTP and use the latest code.

3. Submit the form.

After submission, the system creates your case, and you enter your details.

3.2 Enter your details:

- age/gender

- chief complaint

- urgency level (1-5)

- symptoms list

Submit the form

3.3 Track your case

The page updates your case status and care-team notes.

Possible statuses shown to patients:

- Submitted - request received

- Under review - nurse is reviewing

- Nurse follow-up (watch) - nurse monitoring

- Awaiting doctor review - sent to doctor

- Doctor requested - go to the nursing desk

- Completed - case closed

- Withdrawn / Resolved - closed without full path

3.4 Understand recommendations and alerts

You may see:

- nurse recommendations

- triage level updates

- doctor alerts

Follow the instructions shown on the page.

3.5 Optional patient actions

Depending on status:

- withdraw request

- mark as resolved

Disabled once the case is concluded.

4) Nurse Guide

4.1 Sign in

1. Go to Staff.

2. Log in as a nurse.

3. Access Nurse dashboard.

4.2 Review queue

Check:

- demographics

- symptoms

- urgency

- timestamps

4.3 Make decisions

- accept or override triage

- Add recommendation

- set watch

- forward to the doctor

4.4 Doctor requests

- coordinate patient movement

- Reopen the case if needed

5) Doctor Guide

5.1 Sign in

Log in via the Staff → Doctor dashboard.

5.2 Dashboard tabs

- Under review

- Requested

- Completed/history

5.3 Actions

- review case

- read nurse notes

- Request patient

- mark complete

6) Public Queue Guide

Shows anonymized data:

- pending

- active

- completed

No personal info displayed.

7) Common issues

OTP not received:

- check spam

- resend OTP

Session issues:

- reopen page

Action blocked:

- depends on case status

8) Safety and privacy

- follow staff instructions

- do not share OTP

- queue is anonymized

9) Role summary

Patient: submit → track → follow advice

Nurse: review → guide → forward

Doctor: review → request → complete